

Lab 35 — Linux Permissions, Ownership and User Management

Course: CompTIA Certified A+ Training (Core 1 and Core 2) (TGS-2024048317)

Topic 06: Operating Systems — Core 2, 28% of Core 2

Exam objective: Apply the Linux permission model to users, groups and others using symbolic and numeric notation, and manage users and sudo rights (Core 2 objective 1.11).

Goal

You work the permission model until numeric notation is automatic, then apply it to a real access-control problem: giving a group shared write access to a directory without giving everyone else access. This is exactly what the exam tests and what the job requires.

What you'll produce

A permission notation conversion table and a working shared group directory with verified access control.

Tools and equipment

Killercoda Ubuntu Playground

Browser tools used in this lab

- **Killercoda Ubuntu Playground** — <https://killercoda.com/playgrounds/scenario/ubuntu>



Killercoda Ubuntu Playground — a real Linux machine

Terminal — root@controlplane

```
$ cat /etc/os-release | grep PRETTY
PRETTY_NAME="Ubuntu 22.04.3 LTS"
$ apt-get update -qq && apt-get install -y net-tools dnsutils
Reading package lists... Done
$ ip -brief addr show
lo        UNKNOWN    127.0.0.1/8
eth0      UP            10.244.0.14/24
$ dig +short www.tertiarycourses.com.sg
104.22.35.191
$ chmod 640 ~/applus/logs/system.log && ls -l ~/applus/logs/
-rw-r----- 1 root root 118 Aug 19 09:14 system.log
$ grep -c 'ERROR' ~/applus/logs/system.log
2
$ _
```

1

Open the URL and wait for the prompt. You get root on a real Ubuntu machine — nothing to install.

3

The playground RESETS when idle. If your terminal disappears, reopen it and re-run the setup steps.

Killercoda Ubuntu Playground · <https://killercoda.com/playgrounds/scenario/ubuntu>

Killercoda Ubuntu Playground — the panels and fields this lab uses.

Safety

- Disconnect mains power and hold the power button for 15 seconds before working inside any

machine.

- Wear an anti-static strap connected to an earthed point; hold boards and cards by their edges.
- **Never open a power supply unit or a CRT monitor** — both retain a lethal charge after being unplugged.
- Never puncture, compress or charge a swollen lithium battery; isolate it and follow hazardous disposal procedure.

Step-by-step

1. Open the Killercode playground and examine a file's permission string, identifying the three permission triplets for owner, group and others.

```
bash touch ~/permtest.txt && ls -l ~/permtest.txt
```

2. Record the notation: r equals 4, w equals 2, x equals 1, so rwx is 7, rw- is 6, r-x is 5 and r-- is 4.
3. Build the conversion table for the modes you will actually see: 777, 755, 750, 700, 644, 640 and 600, giving the symbolic form and the typical use of each.
4. Set permissions numerically and verify the symbolic result matches your table.

```
bash chmod 644 ~/permtest.txt && ls -l ~/permtest.txt && chmod 600 ~/permtest.txt && ls -l ~/permtest.txt
```

5. Set permissions symbolically and confirm it produces the same result as the numeric form.

```
bash chmod u=rw,g=r,o= ~/permtest.txt && ls -l ~/permtest.txt
```

6. Record why directories need the execute bit: without x on a directory you cannot enter it or access anything inside, even with read permission.

```
bash mkdir ~/testdir && chmod 644 ~/testdir && (cd ~/testdir 2>&1 || echo 'Cannot enter: no execute bit') && chmod 755 ~/testdir && cd ~/testdir && pwd
```

7. Create two users to demonstrate real access control between accounts.

```
bash useradd -m alice && useradd -m bob && echo 'alice:Pass2026!' | chpasswd && echo 'bob:Pass2026!' | chpasswd
```

8. Create a shared group and add both users to it.

```
bash groupadd support && usermod -aG support alice && usermod -aG support bob && groups alice && groups bob
```

9. Create a shared directory owned by the group with permissions that allow the group to write but exclude everyone else.

```
bash mkdir -p /srv/shared && chown root:support /srv/shared && chmod 770 /srv/shared && ls -ld /srv/shared
```

10. Set the setgid bit so that files created inside inherit the group, which is what makes shared directories actually work in practice.

```
bash chmod g+s /srv/shared && ls -ld /srv/shared
```

11. Verify the access control works: alice can create a file, and the file inherits the support group from the setgid bit.

```
bash su - alice -c 'touch /srv/shared/alice-file.txt' && ls -l /srv/shared
```

12. Grant alice administrative rights through sudo and record why sudo is preferred over logging in as root.

```
bash usermod -aG sudo alice && groups alice && echo 'sudo gives auditable, per-command elevation rather than a permanent root session'
```

Test it — verification

Your conversion table is correct for all seven modes; the execute-bit demonstration shows a directory becoming enterable at 755 but not at 644; /srv/shared is mode 2770 owned by the support group; and alice's file inherits the support group.

Troubleshooting this lab

Symptom	What to check
A command returns "command not found"	Re-run the <code>apt-get install</code> step at the start of the lab — the playground starts with a minimal package set.
The Killercoda terminal has reset	The playground times out when idle. Reopen it and re-run the setup commands from step 1.
A browser tool will not load	Check the URL against <code>labs/tools.md</code> . All four tools run entirely client-side and need no login.
Output differs from the guide	Record what you actually observed — your environment differs from the reference, and explaining the difference is part of the exercise.

Review questions

1. State the exam objective this lab maps to, in your own words.
2. Which single step in this lab would you perform first on a real support call, and why?
3. What evidence would you attach to a support ticket to show this work was completed correctly?
4. Name one thing that would make this procedure fail, and how you would recognise it.

Record your evidence

Complete worksheet.md as you work through this lab and keep it — the Practical Performance assessment mirrors these tasks.
